OpenUBEM European Locations — Engineering Walkthrough & Operational Guide

Step-by-Step Manual for Standards Insertion, Building Stock Enrichment, Procedural Layout Slicing, and BEM Simulation Coupling with GSSCanada

- **Document Version**: `1.0.0-PROD`
- **Location in Repo**: `docs/docs_ACTIVE/europeanLocations/WALKTHROUGH_european_locations.md`
- **Sister MVP Implementation Spec**: `docs/docs_ACTIVE/europeanLocations/MVP_european_locations.md`
- **GSSCanada Reference**: `C:\Users\o_iseri\Desktop\GSSCanada\GSSCanada-main\4J_docs_occ\Step8_docs\IMP_step8\4thJ_08_bemSimulation_IMP.md`
- **Scientific & Algorithmic Provenance**: *Iseri et al. (2025), Energy and Buildings 337, 115620*; `IMP_step8/outputs/floor_layout_generation_report.md`, `IMP_step8/outputs/kbem_ankara_pipeline.py`, `IMP_step8/extracted_scripts/custom_scripts_catalog.json`
- **Core OpenUBEM Docs**: `OpenUBEM_fundamentals.md`, `OpenUBEM_inputs_reference.md`, `OpenUBEM_imputation_methods.md`, `simulated_vs_reconstructed_methodology.md`, `OpenUBEM_debug_References.md`
- **Reusable Figure/Table Assets**: `content/`

> **Document role.** The MVP is the principal technical specification. This walkthrough is the execution layer: ordered tasks, commands, evidence requirements, stop conditions, and an append-only progress log. When scientific wording differs, update the MVP first and make this document point to that decision.
>
> **France scope.** France is included now in residential acquisition/filtering, archetype preparation, neighbourhood/layout construction, IDF/weather work, and controlled baseline physical simulations. France-specific occupant diaries and non-zero occupant-effect schedules remain a future task and must not be counted in the ES/GB/IT 102-archetype/510-run occupant campaign.

> **Implementation-status notice (v1.1 review, 2026-08-22).** The original v1.0 walkthrough below is retained in full for provenance. Several snippets are proposed interfaces rather than runnable calls against OpenUBEM `0.1.0`. Use the code-audited procedure in **Section 9** as the operational authority until the European adapters are implemented and their tests pass.
>
> **Speed/pre-occupant extension.** Section 10 provides the operational Speed runbook, including optional 48/64-way concurrency and the input-map/control pilots required before occupant injection.

---

1. Overview & Operational Pipeline

This walkthrough provides an operational, step-by-step engineering guide for executing European Urban Building Energy Modeling (UBEM) simulations in OpenUBEM. It walks through the end-to-end operational lifecycle: from ingesting CEN/ISO building codes and TABULA archetypes, to procedurally carving multi-dwelling floor plans with unconditioned stairwell cores, assembling watertight EnergyPlus models, and injecting GSSCanada stochastic demographic occupancy diagrams.

```
+-----------------------------------------------------------------------------------------------+
|                        SIX-PHASE EUROPEAN UBEM SIMULATION PIPELINE                            |
+-----------------------------------------------------------------------------------------------+
| Phase 1: Standards Ingestion      -> Load CEN/ISO, CTE, Part L, UNI/TS & TABULA parameter tables |
| Phase 2: Building Enrichment      -> Query OSM footprints & resolve gaps via 4-tier imputation     |
| Phase 3: Procedural Floor Layout  -> Regularize footprints (EdgeTo4) & subdivide dwellings (1x1-4x2)|
| Phase 4: Watertight IDF Assembly  -> Dynamic insulation sizing, internal mass & hydronic heating  |
| Phase 5: GSSCanada Sweep Runner   -> Inject 5-level phi_int schedules & run SLURM HPC cluster     |
| Phase 6: EUI & Gate Verification  -> Calculate simulated vs reconstructed EUI & audit Gates G8.0-16|
+-----------------------------------------------------------------------------------------------+
```

Walkthrough Figure 1. Six execution phases from standards registration to evidence-backed acceptance. Reusable source: `content/walkthrough_figure_1_execution_pipeline.mmd` .

Why dwelling-level zoning matters: The Ankara KBEM study (*Iseri et al., 2025*; 6,458 dwelling units across 277 buildings) demonstrated that building-level aggregation suppresses 75.5% of inter-dwelling energy variance (std dev drops from 63.61 to $15.54\ \mathrm{kWh/m^2a}$) and underestimates peak thermal vulnerability by $3.2\times$. Corner units consume $25\text{--}40\%$ more heating energy than middle units; top-floor units have $15\text{--}25\%$ higher heating demand than mid-floor units. This variance is invisible to coarse building-level models and is the primary scientific justification for the procedural dwelling-subdivision approach in Phase 3.

---

2. Phase 1: Ingesting & Registering European Standards & Physics

2.1 Registering European Envelope & Construction Data

European building envelopes are parameterized against the **TABULA / EPISCOPE** building typology database. Parameter tables are registered in `openubem/data/construction/` : - `tabula_archetypes_es.json` (Spain: 24 archetypes across 6 epochs) - `tabula_archetypes_uk.json` (United Kingdom: 36 archetypes across 8 epochs) - `tabula_archetypes_it.json` (Italy: 42 archetypes across 8 epochs) - `tabula_archetypes_fr.json` (France physical-model registry: count remains `NOT_AUDITED` ; not part of the 102-record occupant registry)

The first three files support the frozen ES/GB/IT occupant campaign. The French file is a current physical-pipeline deliverable: it must be generated from pinned French TABULA/EPISCOPE sources, audited independently, and used for controlled baseline simulations before any France occupant work begins.

TABULA Source Artefact Checksums (for reproducibility): - `tabula-values.xlsx` (4.0 MB): MD5 `7347b2cae3c4d9f5ce78221e9d5fb832` - `tabula-calculator.xlsx` (34.4 MB): MD5 `c99ddc9ffcb6dc0ae7391273d9619e37`

> [!IMPORTANT] TABULA labels the English building stock as `GB` , but survey coverage is limited to **England only** — Scotland, Wales, and Northern Ireland have separate EPCs and building regulations. Use `gb` for the TABULA stock code and retain a separate `uk` survey-fold field in all archetype records.

Example Construction Table Entry (`openubem/data/construction/tabula_archetypes_es.json`):

```
{
  "ES.04.MFH": {
    "country": "ES",
    "epoch_code": "ES.04",
    "year_range": [1980, 2006],
    "building_type": "MFH",
    "description": "Spanish Multi-Family House (NBE-CT-79 Era)",
    "climate_zone": "ES-D3-Madrid",
    "envelope": {
      "u_wall_w_m2k": 1.20,
      "u_roof_w_m2k": 0.90,
      "u_floor_w_m2k": 1.40,
      "u_window_w_m2k": 3.40,
      "g_gl_window": 0.75,
      "infiltration_ach": 0.40
    },
    "thermal_mass": {
      "capacitance_wh_m2k": 50.0,
      "internal_mass_ratio": 1.5
    },
    "hvac": {
      "system_type": "HydronicBaseboard",
      "boiler_efficiency": 0.88,
      "fuel": "NaturalGas",
      "heating_setpoint_c": 20.0,
      "cooling_setpoint_c": 26.0
    }
  }
}
```

2.2 Sizing Opaque Insulation Dynamically (`openubem.idf.opaque_assembly`)

OpenUBEM sizes the insulation thickness (d_{ins}) parametrically so that the composite opaque assembly matches the exact statutory TABULA U-value. Representative wall U-value ranges by epoch (from TABULA / DR06): - **Spain:** Pre-1979 $U_{\text{wall}} \approx 1.4\text{--}1.8$; CTE 2006 $\approx 0.66\text{--}0.95$; CTE 2013 $\approx 0.38\text{--}0.52\ \text{W}/(\text{m}^2\text{K})$ - **UK:** Pre-1918 solid brick ≈ 2.1; Post-1990 ≈ 0.45; Part L 2021 $\approx 0.18\ \text{W}/(\text{m}^2\text{K})$ - **Italy:** Pre-1975 $\approx 1.2\text{--}1.9$; Legge 10/1991 ≈ 0.50; DM 2015 $\approx 0.26\text{--}0.34\ \text{W}/(\text{m}^2\text{K})$

```
# Sizing calculation in openubem/idf/opaque_assembly.py
def calculate_insulation_thickness(
    u_target: float,
    lambda_ins: float = 0.038,     # EPS conductivity [W/(m·K)]
    r_base_layers: float = 0.45,   # Outer brick (0.20m/0.79) + inner plaster (0.015m/0.25)
    r_si: float = 0.13,            # Internal surface resistance (EN ISO 6946)
    r_se: float = 0.04,            # External surface resistance (EN ISO 6946)
) -> float:
    """Calculates continuous insulation thickness to hit exact European U-value."""
    r_target = 1.0 / float(u_target)
    r_needed = r_target - (r_si + r_se + r_base_layers)
    if r_needed <= 0.0:
        return 0.001  # Uninsulated historic masonry baseline (minimum 1 mm layer)
    return round(r_needed * lambda_ins, 4)
```

2.3 Explicit Internal Thermal Mass Injection (`openubem.idf.builder`)

To prevent artificial indoor temperature spikes and correctly represent solid masonry and concrete partitions, OpenUBEM injects `InternalMass` objects into each living zone:

```
# Injection in openubem/idf/builder.py
def inject_european_internal_mass(idf, zone_name: str, floor_area_m2: float, country: str):
    """Injects calibrated internal thermal mass per EN ISO 52016-1 Table B.14."""
    capacitance_map = {
        "ES": 50.0,   # Spain Heavy (Wh/m²·K)
        "IT": 87.0,   # Italy Very Heavy (Wh/m²·K)
        "GB": 32.8,   # UK Medium-Light (Wh/m²·K)
        "EU": 45.0,   # Harmonized European Standard (Wh/m²·K)
    }
    c_m = capacitance_map.get(country.upper(), 45.0)
    mass_surface_area = round(1.5 * floor_area_m2, 2)

    construction_name = f"InternalMass_{country}_Construction"
    if not idf.getobject("CONSTRUCTION", construction_name):
        material_name = f"InternalMass_{country}_Material"
        # 10 cm solid masonry partition equivalent
        idf.newidfobject(
            "MATERIAL",
            Name=material_name,
            Roughness="MediumRough",
            Thickness=0.10,
            Conductivity=0.80,
            Density=2000.0,
            Specific_Heat=c_m * 3600.0 / (0.10 * 2000.0),  # Calibrate c_p to match c_m
            Thermal_Absorptance=0.9,
            Solar_Absorptance=0.7,
            Visible_Absorptance=0.7,
        )
        idf.newidfobject("CONSTRUCTION", Name=construction_name, Outside_Layer=material_name)

    idf.newidfobject(
        "INTERNALMASS",
        Name=f"{zone_name}_InternalMass",
        Construction_Name=construction_name,
        Zone_or_ZoneList_Name=zone_name,
        Surface_Area=mass_surface_area,
    )
```

> [!NOTE] EN ISO 52016-1 Table B.14 defines five standard thermal mass classes: Very Light ($80\ \mathrm{kJ/m^2K}$ / $14\ \mathrm{Wh}$), Light ($110/28$), Medium ($165/45$), Heavy ($260/78$--87), Very Heavy ($370/105\ \mathrm{Wh/m^2K}$). The country-specific values in the `capacitance_map` above are project mapping decisions from these five classes, not exact Table B.14 entries. Their provenance must be documented alongside the archetype parameters.

---

3. Phase 2: Ingesting & Modeling European Building Stocks

3.1 Fetching & Harmonizing OpenStreetMap Footprints

OpenUBEM's acquisition module fetches raw footprints and cleans them into valid UTM planar polygons:

```
from openubem.acquisition.osm_fetcher import fetch_osm_buildings

# Fetch buildings for a district in Madrid, Spain
gdf = fetch_osm_buildings(
    place_query="Chamberí, Madrid, Spain",
    timeout_s=60,
)
print(f"Acquired {len(gdf)} building footprints in Chamberí.")
```

3.2 Resolving Missing Data via the 4-Tier Imputation Cascade

Real-world OpenStreetMap footprints frequently lack `building:levels`, `start_date`, or `height`. Stage 2 resolves these gaps using the zero-fitted priority cascade:

```
from openubem.semantic.imputation import impute_missing

# Impute levels, year_built, and height_m across the European fleet
enriched_gdf = impute_missing(
    gdf,
    target_columns=["levels", "year_built", "height_m"],
    country_code="ES",
    random_seed=42,
)

# Verify provenance tokens
print(enriched_gdf[["building_id", "levels", "levels_provenance", "year_built_provenance"]].head())
```

---

4. Phase 3: Procedural Floor Layout Generation & Dwelling Slicing

4.1 Algorithmic Subdivision Functions (from `kbem_ankara_pipeline.py`)

OpenUBEM incorporates the procedural slicing logic developed in *Iseri et al. (2025)*:

```
from typing import Tuple

def get_grid_division_counts(units_per_floor: int) -> Tuple[int, int]:
    """Determines the (U, V) grid subdivision counts based on target flats per floor."""
    if units_per_floor <= 1:
        return (1, 1)
    elif units_per_floor == 2:
        return (2, 1)
    elif units_per_floor in (3, 4):
        return (2, 2)
    elif units_per_floor in (5, 6):
        return (3, 2)
    else:
        return (4, 2)

def map_construction_vintage(year_built: int) -> str:
    """Classifies building construction year into representative European vintage epochs."""
    if year_built < 1980:
        return "ES.03"  # Pre-NBE-CT-79
    elif 1980 <= year_built < 2007:
        return "ES.04"  # NBE-CT-79 Standard
    elif 2007 <= year_built < 2014:
        return "ES.05"  # CTE-2006 Standard
    else:
        return "ES.06"  # CTE-2013/2019 Standard
```

4.2 Applying Procedural Layout Slicing (`openubem.geometry.layoutGenerator`)

For multi-family typologies (`MFH`, `AB`), `openubem.geometry.layoutGenerator` slices the floor plate into discrete dwelling units and carves out the central circulation core.

```
from openubem.geometry.layoutGenerator import generate_layout
from shapely.geometry import Polygon

# Define a 20m x 15m residential floor plate (300 m² per floor, 4 storeys)
footprint = Polygon([(0, 0), (20, 0), (20, 15), (0, 15)])

zones = generate_layout(
    osm_id="ES_MAD_10482",
    footprint_poly=footprint,
    archetype_id="MidriseApartment",  # Maps to 2x2 quadrant layout + stair core
    num_floors=4,
    floor_to_floor_m=3.0,
)

print(f"Generated {len(zones)} 3D zone dictionaries across 4 storeys.")
for z in zones[:5]:
    print(f" - Zone: {z['name']} | Type: {z['space_type']} | Area: {z['floor_area_m2']:.1f} m² | Z: [{z['z_floor']}, {z['z_
```

4.3 Habitability Sanity Verification (Windowless Unit Test)

Every generated dwelling zone is automatically verified against the exterior facade contact threshold ($L_{\text{ext}} \geq 2.50\ \text{m}$):

```
def verify_habitability(dwelling_poly: Polygon, building_footprint: Polygon) -> bool:
    """Ensures dwelling unit has sufficient exterior facade length for daylight/ventilation."""
    facade_intersection = dwelling_poly.boundary.intersection(building_footprint.boundary)
    ext_length = facade_intersection.length
    if ext_length < 2.50:
        raise ValueError(f"Windowless unit detected! Exterior facade length = {ext_length:.2f} m < 2.50 m")
    return True
```

> [!NOTE] The $2.50\ \text{m}$ threshold originates from IRC Section R303 and Turkish Zoning Law. It is treated as a declared project assumption; its jurisdictional basis must be confirmed for each European stock before the full campaign.

Corridor Width Sizing

For I-shape linear gallery layouts ($L/W \geq 2.0$), the central double-loaded circulation corridor spine width is proportional to building depth, typically $w = 1.20\text{–}1.80\ \text{m}$.

Geometry Fallback Hierarchy

When the target grid fails (degenerate or narrow footprint), the layout generator applies a deterministic cascade: 1. Attempt the assigned grid (2×2, 3×2, etc.) 2. Fall back to next smaller grid ($2 \times 2 \rightarrow 2 \times 1$) 3. Fall back to 1×1 (full floor plate as single zone)

Every fallback records an explicit reason token. From the Ankara validation dataset, 8.3% of buildings (25/277) required fallback, primarily due to narrow footprints ($< 8\ \text{m}$ width) or highly irregular shapes.

Ground-Contact Modeling

The Ankara pipeline tested `GroundDomain:Slab` but abandoned it for `Ground:FcfactorMethod` due to stability issues. The European campaign ground-contact method must be selected, documented, and applied consistently across all archetypes.

4.4 Verifying the Layout Method with OpenUBEM and Grasshopper

Run the following tasks before accepting the procedural layout method:

1. Export a pinned golden set containing a rectangle, rotated rectangle, L-shape, narrow footprint, courtyard footprint, MFH stack, and AB stack in a projected CRS.
2. Process the exact same footprints and requested dwelling counts through Grasshopper and `generate_layout`.
3. Normalize both outputs to GeoJSON or WKT; ignore object order but preserve stable building/floor/zone IDs.

4. Compare dwelling count, circulation count, polygon validity, union area, overlap/gap area, exterior-facade contact, and adjacency.
5. Extrude the clean outputs, create the IDF, reopen it from disk, and verify reciprocal interzone surfaces and zone assignments.
6. Mutate one clean fixture at a time to create an overlap, gap, windowless dwelling, unpaired wall, or invalid fallback; record that the named gate fails.
7. Retain the Grasshopper definition/version, input/output files, OpenUBEM command, normalized comparison table, and preview images.

Use the full `GEO-01` – `GEO-10` acceptance matrix in MVP Section 4.8 and its machine-readable copy at `content/table_4_8_geometry_verification_matrix.csv`. Do not mark parity `PASS` from a visual overlay alone.

---

5. Phase 4: Watertight EnergyPlus IDF Assembly

5.1 Assembling Multi-Zone Geometry & Boundary Conditions

OpenUBEM's IDF builder reads the procedural zone definitions and builds watertight 3D geometry with automated boundary condition matching:

```
from openubem.idf.builder import IDFModelBuilder

builder = IDFModelBuilder(
    osm_id="ES_MAD_10482",
    archetype_code="ES.04.MFH",
    zones=zones,
    country="ES",
)

# Add envelope constructions, window fenestration (WWR = 25%), and PTAC/radiator plant
idf = builder.build()
idf.saveas("output_models/ES_MAD_10482.idf")
print("Watertight multi-zone EnergyPlus IDF successfully assembled.")
```

5.2 Hydronic Radiator & Unconditioned Stairwell Zone Setup

The IDF builder configures: 1. **Dwelling Units**: Conditioned living zones with hydronic radiator heating (`ZoneHVAC:IdealLoadsAirSystem` linked to hot-water boiler curves, $20.0°C$ heating setpoint, $26.0°C$ cooling setpoint where applicable). 2. **Staircase / Corridor Core**: **Unconditioned floating thermal zone** (`Heating Setpoint = None`, `Cooling Setpoint = None`). No `People`, dwelling equipment, or active thermostat objects are assigned to the circulation core unless explicitly justified. 3. **Inter-Zone Party Walls**: Boundary condition `Surface` pointing to adjacent unit / stair zone names with matched reciprocal vertices.

> [!IMPORTANT] **HVAC Plant Modeling Decision (DR07):** Resolving full explicit EnergyPlus hydronic plant loops (boilers, pumps, valves, distribution piping) across 510 multi-zone models introduces severe numerical convergence failures. The campaign standardizes on `ZoneHVAC:IdealLoadsAirSystem` post-processed with natural gas condensing boiler seasonal efficiency curves ($\eta_{\text{seasonal}} = 0.90$, range 0.88–0.94), which preserves identical envelope heat balance while achieving 100% convergence.

5.3 Glazing Parameterization Note

European total solar energy transmittance (g_{gl} per EN 410 / ISO 9050) is mapped to EnergyPlus `WindowMaterial:SimpleGlazingSystem` as $\text{SHGC} = g_{\text{gl}}$. The conversion is valid within ± 0.02 for standard residential glazing. Typical ranges: standard double clear $g_{\text{gl}} = 0.67$–0.75; low-emissivity $g_{\text{gl}} = 0.50$–0.60.

5.4 Natural Ventilation & Infiltration Assumptions

The European campaign maintains a **closed-window assumption** with continuous background infiltration per TABULA methodology. Natural ventilation window opening is not modeled. This is consistent with the Ankara validation study (*Iseri et al.,*

2025) and ensures that the only experimental variable is the occupancy-driven internal gain schedule. Ventilation rates are assigned per country: ES $\mathbf{0.40\ ACH}$, GB $\mathbf{0.59\ ACH}$, IT $\mathbf{0.30\ ACH}$ (see EN 16798-1 Table B.4 equivalence in MVP Section 2.2.1).

---

6. Phase 5: GSSCanada Stochastic Occupancy Ingestion & Parallel Execution

6.1 Generating 5-Level Sensitivity Schedules $\phi_{\text{int}}(t)$

The GSSCanada LLM occupancy pipeline generates 8,760-hour presence curves $g(t)$ from held-out HETUS diaries. OpenUBEM ingests these into the 5-level sensitivity formula:

```
import numpy as np
import pandas as pd

def generate_sensitivity_schedules(
    g_presence_8760: np.ndarray,
    output_csv_dir: str,
    building_id: str,
):
    """Generates 5 external CSV schedules for the sensitivity sweep f in {0.00, 0.15, 0.30, 0.50, 1.00}."""
    mean_g = np.mean(g_presence_8760)
    g_normalized = g_presence_8760 / mean_g

    sweep_factors = [0.00, 0.15, 0.30, 0.50, 1.00]
    generated_files = {}

    for f in sweep_factors:
        # phi_int(t) = (1 - f)*3.0 + f*3.0*g_norm(t)
        phi_int_t = (1.0 - f) * 3.0 + f * 3.0 * g_normalized

        # Verify strict annual energy conservation: 3.0 W/m² * 8760 h = 26.28 kWh/m²
        assert np.isclose(np.mean(phi_int_t), 3.0, rtol=1e-5), "Energy conservation violated!"

        csv_filename = f"{output_csv_dir}/{building_id}_f{int(f*100):03d}.csv"
        df = pd.DataFrame({"phi_int_w_m2": phi_int_t})
        df.to_csv(csv_filename, index=False, header=False)
        generated_files[f] = csv_filename

    return generated_files
```

6.2 Attaching `Schedule:File` in the IDF (Gate G8.13 Conformance)

The generated CSV schedule is wired into the EnergyPlus model via `Schedule:File` with `Interpolate to Timestep = No`:

```
def wire_schedule_file(idf, schedule_csv_path: str, schedule_name: str = "OccupancyInternalGainSched"):
    """Injects Schedule:File object satisfying Pre-Registered Gate G8.13."""
    idf.newidfobject(
        "SCHEDULE:FILE",
        Name=schedule_name,
        Schedule_Type_Limits_Name="Fractional",
        File_Name=schedule_csv_path,
        Column_Number=1,
        Rows_to_Skip_at_Top=0,
        Number_of_Hours_of_Data=8760,
        Column_Separator="Comma",
        Interpolate_to_Timestep="No",  # Strictly enforced per Gate G8.13
    )
```

6.3 Parallel SLURM HPC Execution

Simulations are executed across cluster compute nodes via OpenUBEM's parallel runner (`openubem.simulation.parallel`):

```
# Submit 510-cell campaign array on Concordia Speed SLURM cluster
sbatch --array=1-510 -c 1 --mem=4G --wrap="bash -lc 'python -m openubem.simulation.runner --cell-id \$SLURM_ARRAY_TASK_ID -
```

---

7. Phase 6: Post-Processing, EUI Reconstruction & Gate Audit

7.1 Simulated vs. Reconstructed EUI Calculation

Once EnergyPlus completes, OpenUBEM extracts the 4 simulated physics end-uses and reconstructs the whole-building EUI using national TABULA Table 4 end-use splits:

```
from openubem.results.service_loads import reconstruct_eui

# Simulated results from EnergyPlus SQL / CSV output
simulated_metrics = {
    "heating_eui_kwh_m2": 68.40,
    "cooling_eui_kwh_m2": 14.20,
    "lighting_eui_kwh_m2": 12.50,
    "equipment_eui_kwh_m2": 18.00,
}

# Simulated 4-end-use sum = 113.10 kWh/m²·yr
simulated_total = sum(simulated_metrics.values())

# Reconstruct whole EUI adding DHW, cooking, parasitics
reconstructed_eui, breakdown = reconstruct_eui(
    simulated_eui=simulated_total,
    archetype="MFH",
    country="ES",
)

print(f"Simulated EUI (4 End-Uses):    {simulated_total:.2f} kWh/m²·yr")
print(f"Reconstructed Whole-Bldg EUI:  {reconstructed_eui:.2f} kWh/m²·yr")
print(f" - DHW Share:                  {breakdown['dhw_eui']:.2f} kWh/m²·yr")
print(f" - Cooking Share:              {breakdown['cooking_eui']:.2f} kWh/m²·yr")
print(f" - Parasitics Share:           {breakdown['parasitics_eui']:.2f} kWh/m²·yr")
```

7.2 Pre-Registered Gate Audit Checklist

Every completed simulation cell must satisfy the full validation gate suite before acceptance into the campaign dossier:

Gate	Task	Required evidence	Status before execution
G8.0	Audit `f=0` controls before release of `f>0`	Generated control report	`NOT_RUN`
G8.8	Compare scenario result hashes and values	Scenario comparison report	`NOT_RUN`
G8.9	Mutate one dependency and confirm rerun	Cache mutation report	`NOT_RUN`
G8.10	Reconcile selected total and components	Meter-balance report	`NOT_RUN`
G8.11	Validate required meter names	Parsed `.mdd` and SQL evidence	`NOT_RUN`
G8.12	Reopen IDF and verify schedule path/checksum	Saved-artifact audit	`NOT_RUN`
G8.13	Verify interpolation setting	Saved-IDF audit	`NOT_RUN`
G8.14	Validate manifest against measured files	Manifest schema report	`NOT_RUN`
G8.15	Classify all warnings/errors	Warning taxonomy report	`NOT_RUN`
G8.16	Join schedule provenance to held-out fold	Fold audit report	`NOT_RUN`

Walkthrough Table 1. Gate-execution checklist. Status changes only from generated evidence. Machine-readable copy: `content/walkthrough_table_7_2_gate_checklist.csv` .

7.3 Negative Control Diagnostic Thresholds

Before accepting any European simulation results, verify diagnostic plausibility: - **Reject** if $f = 0.00$ uninjected controls produce heating EUI deviating by $> 50\%$ from published TABULA national brochure benchmarks ($Q_H \approx 80\text{–}180\ \mathrm{kWh/m^2a}$). - **Reject** if adapted OpenUBEM median EUI at $f = 0.00$ differs from TABULA national baseline by $> 25\%$. - **Reject** if pre-1945 uninsulated Spanish/Italian archetypes at $f = 0.00$ yield space heating $< 50\ \mathrm{kWh/m^2a}$.

These are diagnostic flags applied during Q1–Q2 qualification (Section 10), not calibration targets. The pipeline must never alter a TABULA parameter to make a pilot EUI appear more plausible.

7.4 Overheating Assessment (CIBSE TM59 / EN 16798-1)

Where cooling performance is relevant (particularly for UK stock), overheating is assessed using the adaptive comfort model (EN 16798-1 Section 6.2, Annex A) and cumulative Indoor Overheating Degree (IOD). UK-specific compliance requires CIBSE TM59 (2017): - *Criterion A*: Living/bedrooms $\leq 3\%$ of occupied hours exceeding $\Delta T \geq 1\ \mathrm{K}$ operative temperature above comfort limit. - *Criterion B*: Bedrooms $\leq 32\ \mathrm{hours}$ exceeding $26°\mathrm{C}$ between 22:00–07:00.

The Ankara validation study reported IOD peaks up to $0.817°\mathrm{C \cdot h/a}$ at dwelling level versus 0.565 at building level, confirming that zone-level resolution captures 44.6% higher peak overheating exposure.

7.5 Intermittent Heating Factor Preservation

TABULA intermittent heating reduction factors ($F_{\mathrm{red,htr}} \in \{0.80, 0.85, 0.90, 0.95\}$) are applied as scalar transmission multipliers on UA, not as thermostat night-setback schedules. This prevents confounding between heating control patterns and the LLM-generated stochastic occupancy signal (Pre-Registered Ruling `D-S8-2`).

---

8. Diagnostic Error Triage (`OpenUBEM_debug_References.md`)

If an EnergyPlus simulation fails or emits warnings, match the error pattern against `OpenUBEM_debug_References.md` :

Observed pattern	Probable cause	Required action
CTF calculation failed	Unstable resistance-capacitance layer combination	Apply registered CTF-safe construction path and rerun the targeted fixture
Intersecting or degenerate surface	Invalid procedural polygon	Repair or reject geometry and retain original failure evidence
Unmatched interzone surface	Missing reciprocal party-wall face	Rebuild pairing and verify both saved-IDF faces
Temperature out of bounds	Potential mass, load, geometry, or control defect	Classify the cause; never accept from a raw warning count alone

Walkthrough Table 2. Initial EnergyPlus error-triage routes. Machine-readable copy: `content/walkthrough_table_8_1_error_triage.csv`.

---

9. v1.1 Operational Migration Walkthrough (Code-Audited)

This section is the runnable migration guide for OpenUBEM `0.1.0`. It deliberately separates commands that work today from interfaces that must be implemented for Step 8.

9.1 Phase 0 — Establish a Reproducible Baseline

Run all commands from the OpenUBEM repository root. Record the resulting commit and versions before changing the European adapters:

```
git rev-parse HEAD
python --version
python -c "import openubem; print(openubem.__version__)"
python -c "from importlib.metadata import version; print(version('openubem'))"
pytest -q
```

OpenUBEM `0.1.0` does not currently export `openubem.__version__`; the `importlib.metadata` command is the authoritative package-version check. The preceding direct check is retained as a useful packaging diagnostic and is expected to fail until `__version__` is exported.

Also record the EnergyPlus binary path, version, and build identifier. OpenUBEM currently defaults to EnergyPlus **23.1**, whereas some Step 8 research artefacts refer to **9.2**. Choose and pin one campaign version; do not combine `.mdd`, IDF, or result assumptions across versions.

Create an implementation evidence directory under the future campaign output, with one immutable subdirectory per run. Never write verification artefacts into the source-data directory.

9.2 Read the Authorities Before Generating Data

Review these documents in order:

1. `Step8_docs/4thJ_08_bemSimulation.md` — active scientific rulings and unresolved decisions;
2. `Step8_docs/4thJ_08_bemSimulation_val.md` — full G8 and V8 validation contract;
3. this walkthrough and the sister MVP addendum;
4. `IMP_step8/outputs/floor_layout_generation_report.md` and `kbem_ankara_pipeline.py` — geometry references;
5. `IMP_step8/DeepResearch/DR05` – `DR07` — standards/adaptation context.

Do not start the full campaign while geometry, material layers, archetype selection, weather acquisition, or EUI accounting remains an undocumented assumption.

9.3 Build the TABULA Registry as Data, Not Hand-Written Examples

The three JSON files shown in Section 2 do not yet exist. Implement a deterministic conversion command that reads pinned TABULA workbooks and produces:

```
openubem/data/construction/tabula_archetypes_es.json
openubem/data/construction/tabula_archetypes_gb.json
openubem/data/construction/tabula_archetypes_it.json
openubem/data/construction/tabula_archetypes_fr.json
openubem/data/construction/TABULA_PROVENANCE.md
```

Use `gb` for the TABULA stock code and retain a separate `uk` survey-fold field. The generator must emit an exclusions table for refurbishment variants, unclassified records, and any dropped row.

Generate and audit the French physical registry in the same command family, but keep its count and manifest separate. France participates in building/neighbourhood preparation and controlled baseline simulations now; do not create `survey_fold=fr`, France diary assignments, or France `f>0` cells until the future occupant branch is explicitly activated.

Before proceeding, assert:

```
ES count = 24
GB count = 36
IT count = 42
total    = 102
construction-period bands = 22
phi_int_w_m2 = 3.0 for every campaign archetype
FR physical count = independently audited; excluded from total=102
```

The sample JSON in Section 2.1 is illustrative. Values must come from the pinned workbooks/source cells; do not copy the example into production data.

9.4 Use the Existing Acquisition and Imputation APIs Correctly

The current acquisition entry point is `ingest_buildings`, not `fetch_osm_buildings`:

```
from pathlib import Path
import numpy as np

from openubem.acquisition.osm_fetcher import ingest_buildings
from openubem.semantic.imputation import ImputeConfig, impute_missing

raw_gdf = ingest_buildings(
    location="Chamberí, Madrid, Spain",
    radius_m=1000.0,
    output_dir=Path("runtime/eu_pilot/01_acquisition"),
)

audit_cfg = ImputeConfig(
    enabled_tiers=("fusion", "spatial", "ml", "statistical"),
    strict=False,
)
imputed_gdf = impute_missing(
    raw_gdf,
    cfg=audit_cfg,
    targets=("year_built", "levels"),
    rng=np.random.default_rng(42),
)
```

Important differences from the original example:

- the arguments are `cfg`, `targets`, and `rng`, not `target_columns`, `country_code`, and `random_seed`;
- provenance columns are named `provenance_year_built`, `provenance_levels`, etc.;
- `impute_missing` is a standalone router and does not replace the complete `enrich_semantics` pipeline;

- country-aware TABULA mapping must be implemented explicitly before calling `enrich_semantics` with European construction/load/schedule tables.

Run strict mode once on each source dataset to inventory gaps before filling them. Preserve that pre-imputation report alongside the completed dataset.

9.5 Acquire and Validate Actual-Weather Files

The current `epw_manager.fetch_epw` resolves station-based EPW files, but the Step 8 decision requires actual meteorological weather aligned with survey fieldwork rather than an arbitrary TMY. Therefore:

1. determine the fieldwork window for each fold;
2. document the twelve-month selection rule;
3. select and license an AMY source/location;
4. convert or validate the resulting EPW without changing timestamps silently;
5. record its SHA-256 and parsed `LOCATION` header;
6. ensure every `f` level within a fold references the exact same weather checksum.

Do not use the current nearest-station TMY download as a substitute merely to unblock a production result. It may be used only for a clearly labelled geometry/IDF smoke test.

9.6 Add European Geometry Without Replacing the Existing Generator

The current callable is:

```
from openubem.geometry.layoutGenerator import generate_layout

zones = generate_layout(
    osm_id="ES_MAD_10482",
    footprint_poly=footprint,
    archetype_id="MidriseApartment",
    num_floors=4,
    floor_to_floor_m=3.0,
)
```

This call is runnable only for archetypes registered in `MODULE_SPECS`; the current residential specification is DOE/IBC-derived. The Step 8 implementation should add a TABULA-aware adapter or European module specifications while keeping `generate_layout` as the stable geometric entry point.

For every pilot geometry, persist a floor-level audit table with:

```
osm_id, floor_id, requested_dwelling_count, emitted_dwelling_count,
circulation_area_m2, dwelling_area_m2, footprint_area_m2,
area_error_fraction, minimum_facade_contact_m, overlap_area_m2,
gap_area_m2, fallback_reason
```

Reject or explicitly fall back when area error exceeds 1%, a dwelling lacks facade contact, an interzone face lacks a reciprocal partner, or the result is not one zone per dwelling. Treat the `2.50 m` facade-contact threshold as a declared project assumption until its jurisdictional basis is confirmed for each stock.

Before processing a complete neighbourhood, execute the residential-only sample ladder:

1. `S0`: four synthetic fixtures—one each for SFH, TH, MFH, and AB—with geometry and saved-IDF checks;
2. `S1`: 12 observed residential buildings covering simple and irregular footprints, using short design-day simulations;
3. `S2`: 32 observed residential buildings spanning old/new and high/low data completeness, using short-period simulations;
4. `S3`: 96 observed residential buildings balanced across available ES/GB/IT/FR physical strata, using annual controlled baselines;
5. `N1`: only after S0–S3 pass, prepare 500–600 residential buildings;
6. `N2`: optionally scale to 1,000 residential buildings after N1 resource and evidence review.

At the source-inventory boundary, write `excluded_buildings.parquet` with the stable ID and exclusion reason for every non-residential or unresolved-use footprint. Assert that none of those IDs appears in the layout, IDF, or simulation manifests.

9.7 Extend the Existing IDF Builder

The supported fleet builder is `run_step3`; `IDFModelBuilder` from Section 5 is a proposed API and is not present:

```
from pathlib import Path
from openubem.idf.builder import run_step3

idf_manifest = run_step3(
    enriched_gdf,
    schedule_library,
    Path("runtime/eu_pilot/03_idf"),
    n_jobs=1,
    resolution_mode="auto",
)
```

For Step 8, extend `BuildingIDF` / `run_step3` through explicit European configuration rather than patching saved IDFs from the campaign driver. At minimum, the adapter must:

- map TABULA U-values and `g_gl` to the current row schema;
- preserve the opaque-assembly CTF safety behavior while adding traceable material layers;
- emit and test European residential HVAC/ventilation;
- exclude active HVAC and dwelling loads from unconditioned circulation zones;
- bind one independently selected diary to each dwelling;
- record all assumptions and fallback reasons in the IDF manifest.

The existing `build_opaque_assembly` does **not** implement the `calculate_insulation_thickness` example from Section 2.2, and the existing builder does **not** inject the `inject_european_internal_mass` function shown in Section 2.3. Implement and test those behaviors before describing them as current.

9.8 Implement the Step 8 Schedule Adapter

Current OpenUBEM writes DOE `Schedule:Compact` objects. Add a separate, tested target API, for example:

```
# TARGET API — to be implemented; not runnable in OpenUBEM 0.1.0
emit_step8_gain_schedule(
    idf=idf,
    presence_path=presence_path,
    sensitivity_f=f,
    dwelling_zone=zone_name,
    emitted_csv_path=cell_schedule_path,
)
```

The implementation sequence is:

1. read `g(t)` from the Step 7 artefact on disk;
2. validate fold, length, timestamps, finiteness, non-negativity, and `mean(g)>0`;
3. compute `phi_int(t)` for each `f`;
4. assert `mean(phi_int)=3.0 W/m²` within the registered tolerance;
5. emit the campaign-local CSV atomically and hash it;
6. create `Schedule:File` with `Interpolate to Timestep = No`;
7. bind the schedule to the intended dwelling gain object;
8. save the IDF;
9. reopen the saved IDF from disk with an independent audit path and verify filename, checksum, interpolation, and object assignment.

Do not use `Schedule_Type_Limits_Name="Fractional"` for a file containing `phi_int` values in `W/m²`. Either emit a normalized dimensionless schedule with a fixed `3.0 W/m²` design level or define a compatible non-fractional schedule type. Values above `1.0` in a fractional schedule are invalid semantics even when EnergyPlus accepts the file.

Also verify that legacy DOE occupancy/equipment schedules are not simultaneously applying a second temporal modulation to the same experimental gain.

9.9 Build the 510-Row Campaign Table

Generate the campaign table before any simulation:

```
from itertools import product

F_LEVELS = (0.00, 0.15, 0.30, 0.50, 1.00)
cells = [
    (archetype_id, f)
    for archetype_id, f in product(archetype_ids, F_LEVELS)
]
assert len(archetype_ids) == 102
assert len(cells) == 510
```

Validate the matrix:

- exactly 102 rows at each `f`;
- exactly five rows per archetype/weather combination;
- unique deterministic `cell_id`;
- country stock and held-out fold agree with Step 7 provenance;
- all five levels share IDF physics, diary selection, and weather, differing only in the registered `f` transformation;
- `f=0` is scheduled first and blocks publication of its four matching `f>0` results until read successfully.

The `f=0` schedule is the flat `3.0 W/m²` endpoint of the same transformation. It should follow the same emission and assignment path so that the control is not constructed by different code.

9.10 Execute Locally, Then Wrap for SLURM

The existing local execution API is:

```
from pathlib import Path
from openubem.simulation.parallel import run_neighbourhood

sim_manifest = run_neighbourhood(
    idf_manifest=idf_manifest,
    enriched_gdf=enriched_gdf,
    sim_root=Path("runtime/eu_pilot/04_simulation"),
    n_jobs=4,
    force_rerun=False,
)
```

This API uses joblib and returns `04_simulation_manifest.parquet`. It is not the `--cell-id` CLI shown in Section 6.3. Build a thin GSSCanada campaign command that accepts one immutable cell specification, invokes OpenUBEM, and writes one `manifest.json`; then submit that command with SLURM.

Before the 510-cell array:

1. run one `f=0` cell locally;
2. run the same cell twice and establish reproducibility;
3. run one non-zero `f` with the same physical inputs;
4. prove scenario differentiation;
5. change the schedule and prove the cache invalidates;
6. run the full gate mutation suite on a disposable pilot cell;
7. only then submit the complete array.

Use a dependency digest for resume decisions. The current `.end` + `.sql` completion check alone cannot satisfy G8.9.

9.11 Choose One EUI Accounting Path

The original `reconstruct_eui(...)` example is not a current API. Current OpenUBEM exposes `reconstruct_frame(...)` and defaults reconstruction off because Phase E physically models service loads.

For a four-end-use experiment, use the existing API only after providing European coefficients and explicitly disabling corresponding physical loads:

```
from openubem.results.service_loads import load_coefficients, reconstruct_frame

coeffs = load_coefficients(european_coefficients_path)
reported = reconstruct_frame(results_df, coeffs=coeffs, force=True)
```

Otherwise, keep physical mode and do not reconstruct. In both cases, report the simulated end uses, reconstructed end uses, floor-area denominator, coefficient checksum, and `eui_accounting_mode`. A run containing both physical service loads and a reconstruction uplift is invalid.

9.12 Run the Complete Gate and Mutation Audit

Replace the fixed `[x] PASS` labels in Section 7.2 with generated statuses. The audit order is:

1. assert expected campaign rows and file paths (V8.a–V8.b);
2. verify `f=0` availability and ordering (G8.0);
3. validate runtime status and warning kinds (G8.15/V8.f);
4. verify meter names and energy balance (G8.10–G8.11);
5. independently reopen saved IDFs and verify schedule value, assignment, and interpolation (G8.12–G8.13);
6. verify manifest fields from measured files/platform (G8.14);
7. join against Step 7 provenance and verify held-out folds, failing on missing fold (G8.16/V8.g);
8. compute reproducibility, peak, band, differentiation, and cache gates (G8.1–G8.9);
9. run every registered mutation and record which gates fall;
10. run the null mutation and confirm that none fall.

The generated gate report must include `status ∈ {PASS, FAIL, BLOCKED, NOT_RUN}`, evidence paths, observed values, thresholds, and failure reasons. `READY`, `implemented`, or a prose checklist is not a substitute for `PASS`.

9.13 Minimum Test Matrix

Add dedicated tests rather than overloading the North American validation suite:

```
tests/test_eu_tabula_loader.py
tests/test_eu_construction_sets.py
tests/test_eu_dwelling_layout.py
tests/test_eu_dwelling_layout_grasshopper_parity.py
tests/test_eu_residential_filter.py
tests/test_eu_sample_group_ladder.py
tests/test_eu_internal_mass.py
tests/test_eu_hvac.py
tests/test_step8_schedule_file.py
tests/test_step8_campaign_matrix.py
tests/test_step8_manifest.py
tests/test_step8_cache_invalidation.py
tests/test_step8_gates.py
tests/test_step8_gate_mutations.py
```

The physical smoke matrix must contain ES, GB, IT, and FR; each residential type; at least one historic and one modern band; simple and non-convex footprints; and the selected baseline weather files. The occupant-schedule matrix contains ES/GB/IT only until the France occupant branch is activated. Include negative tests for non-residential leakage into the registry, Grasshopper/OpenUBEM parity drift, zero-mean/NaN schedules, wrong folds, missing manifests, duplicate cell IDs, schedule reassignment, `Interpolate=Yes`, invalid meter names, altered weather, and double-counted EUI.

9.14 Operational Stop Conditions

Stop the campaign and mark it **BLOCKED** when any of the following occurs:

- TABULA counts/provenance do not reproduce 24/36/42;
- an unresolved assumption changes geometry, construction, weather, or EUI accounting;
- `f=0` fails to run or cannot be read for a matching archetype/weather;
- annual gain conservation fails;
- a country is assigned a fold that did not hold it out;
- a saved IDF cannot independently prove schedule file and object assignment;
- a cache survives an input dependency change;
- an EnergyPlus meter is missing/zero-filled or end-use balance exceeds the registered tolerance;
- any severe/fatal error occurs;
- a validation gate has not been observed failing its designated mutation;
- the scorer reads fewer cells than the campaign manifest declares.

9.15 Expected Handoff Artefacts

A completed Step 8 run should provide:

```
outputs_step8/
├── inputs/
│   ├── archetype_registry.json
│   ├── archetype_parameter_provenance.md
│   ├── schedule_registry.parquet
│   └── weather_registry.json
├── campaign/
│   ├── campaign_cells.parquet
│   └── campaign_config.json
├── archetypes/
├── cells/<cell_id>/
│   ├── manifest.json
│   ├── model.idf
│   ├── gain_schedule.csv
│   ├── eplusout.sql
│   ├── eplusout.err
│   └── results.parquet
├── control/
│   └── f000_read_before_injected.parquet
├── validation/
│   ├── gate_results.parquet
│   ├── mutation_coverage.parquet
│   └── warning_kinds.parquet
├── agg_annual.csv
├── agg_monthly.parquet
└── agg_hourly.parquet
```

The final report must state that G8.1–G8.4 are reproducibility gates rather than measured-accuracy gates, identify `GB` as England-limited TABULA stock, report effects within fold across all five `f` values, name weather years in absolute cross-fold comparisons, and distinguish design assumptions from verified simulation evidence.

---

10. Speed HPC Execution and Pre-Occupant Pilot Runbook

10.1 Select an Execution Profile

Use the existing parallel-processing reference for the separation between parallel IDF preparation and parallel EnergyPlus execution. Use the official Speed manual for current cluster policy.

Set one campaign throttle explicitly:

```
# Conservative default from the established OpenUBEM runbook
export SPEED_MAX_CONCURRENT=32

# Expanded option requested for Step 8; use only after Q1/Q2 pass and approval is recorded
export SPEED_MAX_CONCURRENT=64

# Replace with the account authorized for this campaign
export SPEED_ACCOUNT="replace_with_authorized_account"
```

Recommended values are `4`, `8`, `16`, `32`, `48`, or `64`. Each running task uses one CPU, so `%64` can use more than 32 CPUs simultaneously across several nodes. It does not make one EnergyPlus simulation use 64 CPUs, and it does not guarantee that 64 tasks will start immediately.

Before selecting `48` or `64`, record:

```
sinfo -p ps --long --Node
squeue -u "$USER"
```

These are read-only scheduler checks. Submit all compute through `sbatch`; never execute EnergyPlus on `speed-submit`.

10.2 Create the Pre-Occupant Audit Figure

Before IDF generation, produce a four-panel map matching the analytical intent of `IMP_step8/resources/Screenshot_3.png`:

```
panel_a_construction_period.png
panel_b_epc_availability.png
panel_c_building_function_typology.png
panel_d_construction_material_set.png
preinjection_input_audit_4panel.png
preinjection_input_audit_counts.csv
preinjection_input_audit.gpkg
```

The plotting input must include stable building IDs and these minimum columns:

```
building_id, geometry, country_stock_code, construction_period,
epc_availability, building_function, residential_typology,
observed_material, assigned_construction_set,
provenance_year_built, provenance_material, data_quality_flag
```

For each panel, assert in code that:

```
mapped feature count = source feature count
sum(category counts) = mapped feature count
missing values appear as an explicit category
no spatial join creates or drops duplicate building_id values
```

Use observed and imputed values as separate layers or visual encodings. Do not color an imputed construction material as if it were observed without exposing its provenance.

Keep non-residential and unresolved-use footprints in the audit source so exclusions remain visible, but render them as grey/hatched context and write their IDs to `excluded_buildings.parquet`. The residential modelling registry must satisfy:

```
assert set(excluded["building_id"]).isdisjoint(layout_manifest["building_id"])
assert set(excluded["building_id"]).isdisjoint(idf_manifest["building_id"])
assert set(excluded["building_id"]).isdisjoint(simulation_manifest["building_id"])
```

Use the neighbourhood concept in `content/figure_neighbourhood_residential_typologies.png` only to communicate target scale and morphology. It is illustrative and cannot replace the measured four-panel input audit.

10.3 Build the Qualification Cases

Create five immutable lists:

```
q1_smoke_cells.tsv       # 4 cells: one ES, one GB, one IT, one FR physical baseline
q2_pilot_cells.tsv       # target 32 cells: 4 × 4 typologies × 2 age extremes
q3_control_cells.tsv     # 102 cells, f=0 only
q4_injected_cells.tsv    # 408 cells, f=0.15/0.30/0.50/1.00
fr_baseline_cells.tsv    # one controlled baseline per accepted FR physical archetype
```

Each TSV row must provide, by explicit path rather than directory inference:

```
cell_id<TAB>idf_path<TAB>epw_path<TAB>gain_csv_path<TAB>manifest_path
```

Validate the lists before upload:

```
assert len(q1) == 4
assert len(q2) == 32
assert len(q3) == 102
assert len(q4) == 408
assert fr_baseline["country_stock_code"].eq("fr").all()
assert fr_baseline["sensitivity_f"].eq(0.0).all()
assert q3["sensitivity_f"].eq(0.0).all()
assert set(q4["sensitivity_f"]) == {0.15, 0.30, 0.50, 1.00}
assert not campaign["cell_id"].duplicated().any()
```

Q1–Q3 and the separate France baseline use a constant `3.0 W/m²` gain emitted through the final Step 8 `Schedule:File` adapter. They do not ingest a stochastic occupant diary. Q4 is ES/GB/IT only until the future France occupant branch is accepted.

10.4 Prepare the Speed Directory

Keep the campaign under scratch and make the layout deterministic:

```
export STEP8_REMOTE="/speed-scratch/$USER/openubem/step8_europe"
mkdir -p "$STEP8_REMOTE"/{bin,config,idfs,weather,schedules,manifests,out,logs,harvest}
mkdir -p "/speed-scratch/$USER/tmp"
```

Upload the pinned EnergyPlus Ubuntu 20.04 distribution, campaign inputs, cell lists, and Step 8 scripts from the local workstation. Then compare local and remote SHA-256 registries before submission.

`/speed-scratch` is active storage: the Speed manual states that inactive files may be automatically cleaned after 90 days. Harvest durable outputs to the local project immediately after each qualification stage.

10.5 Step 8 Array Script Template

Create a Step 8-specific batch script rather than using `submit_fleet_t08.sbatch` unchanged:

```
#!/bin/bash
#SBATCH --partition=ps
#SBATCH --nodes=1
#SBATCH --ntasks=1
#SBATCH --cpus-per-task=1
#SBATCH --mem=6G
#SBATCH --time=02:00:00
#SBATCH --job-name=step8_eu
#SBATCH --output=/speed-scratch/%u/openubem/step8_europe/logs/%x_%A_%a.log

set -euo pipefail

: "${CELL_LIST:?CELL_LIST is required}"
: "${STEP8_REMOTE:?STEP8_REMOTE is required}"

ROW=$(sed -n "${SLURM_ARRAY_TASK_ID}p" "$CELL_LIST")
if [ -z "$ROW" ]; then
    echo "No cell at row ${SLURM_ARRAY_TASK_ID}: $CELL_LIST" >&2
    exit 2
fi

IFS=$'\t' read -r CELL_ID IDF EPW GAIN_CSV SOURCE_MANIFEST <<< "$ROW"
for REQUIRED in "$IDF" "$EPW" "$GAIN_CSV" "$SOURCE_MANIFEST"; do
    [ -f "$REQUIRED" ] || { echo "Missing input: $REQUIRED" >&2; exit 3; }
done

EP_DIR=$(ls -d /speed-scratch/$USER/openubem/tools/EnergyPlus-23.1.0-*Ubuntu20* | head -1)
OUTDIR="$STEP8_REMOTE/out/$CELL_ID"
WORKDIR="${TMPDIR:-/speed-scratch/$USER/tmp}/${SLURM_ARRAY_JOB_ID}_${SLURM_ARRAY_TASK_ID}"
mkdir -p "$OUTDIR" "$WORKDIR"

cp "$IDF" "$WORKDIR/in.idf"
cp "$EP_DIR/Energy+.idd" "$WORKDIR/Energy+.idd"
cd "$WORKDIR"
"$EP_DIR/ExpandObjects"
[ -f expanded.idf ] && RUN_IDF="$WORKDIR/expanded.idf" || RUN_IDF="$WORKDIR/in.idf"

set +e
"$EP_DIR/energyplus" -w "$EPW" -d "$WORKDIR" "$RUN_IDF"
RC=$?
set -e
echo "$RC" > "$WORKDIR/task.rc"

for FILE in eplusout.sql eplusout.err eplusout.end eplusout.eio eplusout.mdd task.rc; do
    [ -f "$WORKDIR/$FILE" ] && cp "$WORKDIR/$FILE" "$OUTDIR/$FILE"
done
cp "$IDF" "$OUTDIR/model.idf"
cp "$GAIN_CSV" "$OUTDIR/gain_schedule.csv"
cp "$SOURCE_MANIFEST" "$OUTDIR/source_manifest.json"

exit "$RC"
```

This is a target Step 8 template and must receive a shellcheck/smoke review before use. Unlike the generic script, it chooses the EPW per cell, captures a failing return code despite `set -e`, stages repeated I/O under `$TMPDIR`, and retains meter/IDF/schedule evidence required by G8.10–G8.14.

10.6 Run Q1 and Q2 Before the Full Control

Submit the four-country physical smoke test:

```
Q1_JOB=$(sbatch --parsable \
  --account="$SPEED_ACCOUNT" \
  --array=1-4%4 \
  --export=ALL,STEP8_REMOTE="$STEP8_REMOTE",CELL_LIST="$STEP8_REMOTE/config/q1_smoke_cells.tsv" \
  "$STEP8_REMOTE/bin/step8_cell.sbatch")
echo "$Q1_JOB"
```

After harvest, require 4/4 success, zero severe/fatal errors, meter availability, and correct checksums. Then submit the target 32-cell stratified physical pilot:

```
Q2_JOB=$(sbatch --parsable \
  --account="$SPEED_ACCOUNT" \
  --array=1-32%16 \
  --export=ALL,STEP8_REMOTE="$STEP8_REMOTE",CELL_LIST="$STEP8_REMOTE/config/q2_pilot_cells.tsv" \
  "$STEP8_REMOTE/bin/step8_cell.sbatch")
echo "$Q2_JOB"
```

Do not promote to Q3 merely because EnergyPlus returned zero. Check geometry, construction values, saved-IDF schedule assignment, warning kinds, meter balance, denominator area, service-load mode, and result finiteness.

10.7 Run More Than 32 Controls Concurrently

After Q1/Q2 approval, submit the 102 controls with the selected throttle:

```
: "${SPEED_MAX_CONCURRENT:=32}"
case "$SPEED_MAX_CONCURRENT" in
  4|8|16|32|48|64) ;;
  *) echo "Unsupported throttle: $SPEED_MAX_CONCURRENT" >&2; exit 2 ;;
esac

Q3_JOB=$(sbatch --parsable \
  --account="$SPEED_ACCOUNT" \
  --array="1-102%${SPEED_MAX_CONCURRENT}" \
  --export=ALL,STEP8_REMOTE="$STEP8_REMOTE",CELL_LIST="$STEP8_REMOTE/config/q3_control_cells.tsv" \
  "$STEP8_REMOTE/bin/step8_cell.sbatch")
echo "$Q3_JOB"
```

With `SPEED_MAX_CONCURRENT=64`, SLURM may run up to 64 one-CPU EnergyPlus controls at once—more than 32 CPUs across multiple nodes—while preserving one CPU per simulation.

10.8 Enforce Control Audit Before Occupant Runs

Submit a single audit job after Q3. Its script must harvest/recount the 102 controls, execute the pre-occupant acceptance gates, write `q3_control_gate_report.json`, and exit non-zero on any missing or failed requirement:

```
Q3_AUDIT_JOB=$(sbatch --parsable \
  --account="$SPEED_ACCOUNT" \
  --dependency="afterok:${Q3_JOB}" \
  --partition=ps --cpus-per-task=1 --mem=8G --time=01:00:00 \
  --export=ALL,STEP8_REMOTE="$STEP8_REMOTE" \
  "$STEP8_REMOTE/bin/step8_control_audit.sbatch")
echo "$Q3_AUDIT_JOB"
```

Only then submit the 408 occupant-modulated cells:

```
Q4_JOB=$(sbatch --parsable \
  --account="$SPEED_ACCOUNT" \
  --dependency="afterok:${Q3_AUDIT_JOB}" \
  --array="1-408%${SPEED_MAX_CONCURRENT}" \
  --export=ALL,STEP8_REMOTE="$STEP8_REMOTE",CELL_LIST="$STEP8_REMOTE/config/q4_injected_cells.tsv" \
  "$STEP8_REMOTE/bin/step8_cell.sbatch")
echo "$Q4_JOB"
```

Do not combine Q3 and Q4 in a single 510-row array: array ordering is not a scientific dependency, and `f>0` tasks could start before all controls have been read.

10.9 Monitor and Measure From the Workstation

Use short, read-only queries from the local workstation:

```
squeue -j "$Q3_JOB,$Q3_AUDIT_JOB,$Q4_JOB"
sacct -j "$Q3_JOB" --format=JobID,State,Elapsed,AllocCPUS,MaxRSS,ExitCode
sacct -j "$Q4_JOB" --format=JobID,State,Elapsed,AllocCPUS,MaxRSS,ExitCode
```

Record the raw `sacct` output in the campaign evidence. Do not infer 64-way execution from the submitted throttle; compute actual concurrency and CPU use from scheduler records.

10.10 Harvest Before Cleanup

For Q1, Q2, Q3, and Q4 separately:

1. copy all retained outputs and SLURM logs back to a new local evidence directory;
2. regenerate SHA-256 hashes locally;
3. compare expected cell IDs with harvested cell IDs;
4. build the OpenUBEM/Step 8 manifests;
5. execute gate scoring locally from the harvested immutable files;
6. archive the audit maps and category counts with the simulation report;
7. only after validation and backup, remove the corresponding remote output directory.

Do not apply the generic T08 cleanup before G8.10–G8.14 evidence is harvested. In particular, deleting `.mdd`, the saved IDF, or the emitted gain CSV too early makes independent meter and schedule validation impossible.

10.11 Promotion Checklist

The `f>0` occupant campaign is authorized only when all boxes are backed by generated evidence:

```
[ ] Four-panel input-audit map and exact category counts reviewed
[ ] Q1: 4/4 ES/GB/IT/FR physical smoke cases successful, zero severe/fatal
[ ] Q1 repeated cell passes reproducibility gates
[ ] Q2: target 32/32 physical cases successful or exclusions explicitly approved
[ ] Geometry, envelope, HVAC, weather, and service-load mode validated
[ ] Constant 3.0 W/m² baseline verified from saved IDFs
[ ] Q3: 102/102 controls harvested and independently recounted
[ ] FR baseline manifest is complete and kept separate from the 510-case occupant denominator
[ ] G8.0 and meter/schedule/manifest checks pass
[ ] Mutation tests demonstrate the relevant gates failing
[ ] Q3 audit job exits zero and unlocks Q4
[ ] Selected `%32`/`%48`/`%64` throttle and approval recorded
```

If any item is missing, keep Q4 **BLOCKED**. Faster cluster execution must never shorten the scientific validation sequence.

---

11. Task Ledger and Progress Log

This section is the operational index. Detailed scientific requirements remain in the MVP; implementation sessions update the status and append a dated evidence row here.

Task group	Scope	Initial status	Required promotion evidence
`T-DOC`	MVP/walkthrough roles, captions, reusable assets	`LOCAL_PASS`	Link/caption/PDF checks
`T-FR-PHYS`	French residential registry, layouts, IDFs, weather, baseline simulations	`NOT_RUN`	Audited registry and FR-B evidence bundle
`T-FR-OCC`	French held-out diaries and occupant-effect sweep	`BLOCKED` / future scope	Approved diary/fold/weather contract
`T-FILTER`	Residential-only registry and explicit non-residential exclusions	`NOT_RUN`	Exclusion audit and manifest-disjointness tests
`T-GEO`	`GEO-01` – `GEO-10`, including Grasshopper parity	`NOT_RUN`	Generated comparison and mutation reports
`T-S0-S3`	4/12/32/96-building sample groups	`NOT_RUN`	Per-stage case accounting and resource report
`T-N1`	500–600-building residential neighbourhood	`NOT_RUN`	S0–S3 approval and audited N1 manifest
`T-N2`	Optional scale-up to 1,000 residential buildings	`NOT_RUN`	N1 evidence and explicit capacity approval
`T-Q1-Q4`	Four-country physical smoke and ES/GB/IT occupant campaign	`NOT_RUN`	Stage-specific retained evidence

Walkthrough Table 3. Living task ledger. `BLOCKED` on `T-FR-OCC` describes the intentionally deferred occupant-input dependency; it does not block French physical-model work. The append-only CSV schema is `content/walkthrough_progress_log.csv`.

11.1 Append-Only Progress-Log Rules

For every material attempt, append one row containing UTC date, repository commit, work package/task ID, status, exact command, evidence path, decision or blocker, and one next action. Use only `DOCUMENTED`, `IMPLEMENTED_NOT_TESTED`, `LOCAL_PASS`, `SPEED_SUBMITTED`, `SPEED_COMPLETE_UNAUDITED`, `ACCEPTED`, `NOT_RUN`, or `BLOCKED`. Never overwrite a failed attempt with a later pass; append the correction as a new row.

Date	Commit	Task	Status	Command / evidence	Decision or blocker	Next action
2026-08-23	`e04f42a` + dirty-tree caveat	`T-DOC`	`IMPLEMENTED_NOT_TESTED`	Updated active Markdown and `content/` assets	PDF/link/caption verification pending	Run documentation validation and record outcome
2026-08-23	`e04f42a` + dirty-tree caveat	`T-DOC`	`LOCAL_PASS`	`git diff --check`; relative-link/SVG/CSV validation; `scripts/convert_docs_to_pdf.py`; headless SVG previews	Both PDFs regenerated; repaired figures visually inspected	Begin CP0 / EU-01–EU-02 local implementation slice

Walkthrough Table 4. Human-readable progress-log view. The CSV remains the machine-readable append-only record.